



PeopleIntel

Complete Project & Code Guide

COMPANY · DESIGNATION · PERSON

A lead-research dashboard. You upload a CSV of contacts, and it gives you back researched profiles — who each person is, what they do, what they are currently talking about, and what their company does.

Written in plain language · covers the whole project and every code file

Generated 25 August 2026

Contents

1	What this project is	9	Two statuses, not one
2	The problem it solves	10	The technology used
3	How it works, start to finish	11	How to run it
4	The 9 steps	12	Settings and API keys
5	A tour of the 8 screens	13	What is in the database right now
6	The code, file by file	14	Known limits and what is not built
7	The database explained	15	Glossary of terms
8	The trust rules (the heart of it)	16	Quick file reference

1. What this project is

PeopleIntel is a small web app that researches your sales contacts for you.

Imagine you export a list of 25 people from Apollo (a sales-contact tool). You get a spreadsheet with names, job titles, emails and company names. That spreadsheet tells you almost nothing useful — it does not tell you what these people actually care about right now, or what their company really does.

PeopleIntel takes that spreadsheet and turns each row into a *profile page*:

- **Who they are** — name, job title, email, LinkedIn, location, company.
- **What they've been saying** — their public LinkedIn posts.
- **What the web says about them** — articles, interviews, conference talks.
- **What they're interested in** — short "interest chips", each one backed by real evidence.
- **What their company does** — a description pulled from the company's own website.

The whole thing is built around three ideas the app calls its three pillars: **Company · Designation · Person**. Everything on screen answers one of those three questions.

The single most important idea in this project: the app would rather show you *nothing* than show you something that might be wrong. Almost every design decision in the code comes from that one rule. Section 8 explains it fully.

It is a Python-only project

There is no JavaScript framework, no React, no Node.js. Pages are plain HTML generated on the server. Buttons are plain HTML forms. This makes it very easy to run — install Python packages, start the server, done.

2. The problem it solves

A sales export gives you rows of data. It does **not** give you a reason to talk to someone. Three specific problems:

Problem	What PeopleIntel does about it
The data has holes. Some rows have no email, no LinkedIn, no job title.	It detects the gaps per contact and shows exactly which ones are missing, separating "we don't know who this is" from "we haven't looked them up yet".
Job titles go stale. The title in the file may be a year out of date.	When research finds a different title on the web, it keeps <i>both</i> and flags the mismatch instead of silently overwriting or silently trusting the file.
You have no conversation starter.	It collects what the person actually posted and what the web published about them, then derives interest chips from that text — each chip hover-able to see its source.

In the sample data of 25 contacts, **three out of the four** researched people had a different job title on the web than the one in the CSV. That is how fast this data ages.

3. How it works, start to finish



Step by step, in plain words

- You upload a CSV.** The app figures out the file's separator (comma, semicolon, tab or pipe), its text encoding, skips any junk lines above the real header, repairs a broken header row, and matches the column names to the fields it understands.
- Each row becomes a Person, and each unique company becomes a Company.** If a contact was already imported before (matched on their Apollo Contact Id), the existing record is *updated* rather than duplicated — so re-uploading a corrected export does not leave you with three copies of everybody.
- The app works out what is missing** for each contact and gives them a status.
- Research runs** — either for everyone via the command line (`run_pipeline.py`) or for one person via a *Research* button on their page. Research calls a web-search API (Firecrawl) and a language model.
- Everything found is stored with its source and the time it was fetched**, so every fact on screen is clickable back to where it came from.
- You browse the results** on the dashboard, per-person pages, company pages and the reports page.

Research is deliberately manual. Nothing runs automatically on a schedule. Every research run costs API credits, so it only happens when you ask for it. The *Research* button is not even offered for a contact who already has results — re-running is a deliberate command-line action (`--force`), not something a stray double-click can trigger.

4. The 9 steps

The project describes itself as nine steps. Here is each one in plain language, with the file that does the work.

#	Step	File	What actually happens
1	CSV Upload	<code>app/importer.py</code>	Reads the file however it arrives — any separator, any common encoding, junk lines above the header, even a header row where 17 columns are all wrongly named "First Name".
2	Parsing & Normalisation	<code>app/importer.py</code>	Turns the 75 columns of an Apollo export into the ~25 fields the screens actually use. Column names are matched by alias, so "Company", "Account Name" and "Associated Company" all mean the same thing.
3	Missing Data Enrichment	<code>app/models.py</code> , <code>app/resolve.py</code>	Works out which important fields are blank. <code>resolve.py</code> can go and find a missing email or LinkedIn URL on the web — but only when it can prove the find belongs to this person.
4	LinkedIn Activity	<code>app/research.py</code>	Finds public LinkedIn post links through a web search index. LinkedIn itself is <i>never</i> logged into or fetched. You can also paste posts in by hand.
5	Google / Web Research	<code>app/research.py</code>	Searches the web from two angles (person + company, person + job title) and keeps only results it can tie to this specific person.
6	Interest Detection	<code>app/interests.py</code>	Sends the collected evidence to a language model and gets back up to 5 interest chips plus a one-line "current focus" sentence.
7	Company Research	<code>app/research.py</code>	Finds a description of what the company does, preferring the company's own website.
8	Dashboard	<code>app/main.py</code> , <code>templates/</code>	All the web pages. FastAPI routes plus Jinja2 HTML templates.
9	Reports & Export	<code>templates/reports.html</code>	Breakdowns by seniority, industry and country, plus the list of contacts whose job title has gone stale.

5. A tour of the 8 screens

Every page shares the same layout: a top bar with search, a left sidebar with eight links, and the content in the middle.

Home /

The main dashboard. On the left: the CSV upload box, a row of statistic tiles, a short people list, and the selected person's company card. On the right: the full profile of one person — their details, their interest chips, their LinkedIn activity, and their web findings side by side. If you have not picked anyone, it automatically shows the best-researched contact so the page is never empty.

CSV Upload /upload

Upload a file and get a report back: how many rows were read, how many imported, how many updated an existing contact, how many failed, how many columns were recognised, and a list of *notes*. The notes are important — they tell you things like "read as semicolon-separated, not comma", "skipped 2 lines above the header", or "header row was unusable in 17 of 75 columns and was repaired". Nothing is fixed silently.

People /people

The full contact list with a search box and a status filter. Each row shows the person, their company, both status badges, and a *Research* button if they have not been researched yet.

Person page /person/<slug>

One contact in full. Their details, the gaps in their record, their interest chips with the "Current Focus" line, their LinkedIn activity panel (with a form to paste in a post by hand), and their web findings — every finding showing the site it came from and when it was fetched. If their job title on the web differs from the CSV, that mismatch is flagged here.

Companies /companies and Company page

All the companies pulled out of the CSV, searchable by name or industry. A company page shows its details (industry, size, revenue, funding, technologies, keywords, location) and its researched description with a link to the source. The logo is the site's favicon.

Enrichment /enrichment

A worklist. Every contact with the list of what is missing, sorted so the most incomplete records are at the top. Critical gaps (which hold a contact at "Needs Enrichment") are shown separately from optional gaps (which do not).

Research /research

A table of how much has been found for each contact: number of LinkedIn activities, web findings and interest chips. Sorted best-researched first.

Reports [/reports](#)

Breakdowns of your contact list by seniority, by industry and by country — plus the stale-designation list, showing every contact whose job title in the file disagrees with the one found on the web, with both values side by side.

Settings [/settings](#)

Shows whether your API keys are configured, which language-model endpoint and model are in use, and whether a `.env` file was found (and exactly where it looked for it). It shows only whether a key is *present*, never the key itself.

6. The code, file by file

About 2,000 lines of Python and 1,300 lines of HTML templates. Here is every file and what it is for.

app/main.py (360 lines) — the web server

This is the front door. It defines every URL the app answers.

- **Page routes:** `/`, `/people`, `/person/<slug>`, `/companies`, `/company/<id>`, `/enrichment`, `/research`, `/reports`, `/settings`, `/upload`.
- **Action routes (forms):** upload a CSV, add a LinkedIn activity by hand, delete one, and start a research run for one person.
- **Display helpers** used inside the templates: `relative_time` turns a date into "2 days ago"; `domain` pulls "nelson.com" out of a URL; `money` turns 4200000 into "\$4.2M"; `short_date` formats "24 Aug 2026".
- **The `stats()` function** counts everything the tiles show: total contacts, how many need enrichment, how many were researched, how many research runs failed, how many companies have a description, how many have a stale job title.

A small but real security detail. The Research button sends a `back` field saying which page to return to. The code only honours a value that starts with a single `/` — otherwise someone could craft a link that bounced your users off to another website through your own app.

app/models.py (441 lines) — the shape of the data

Defines the five things the app stores and the rules attached to them. There are five tables: **Company**, **Person**, **LinkedInActivity**, **WebFinding** and **Interest**. Section 7 covers the fields; the interesting part is the logic that lives here:

- `missing_critical()` — the gaps that matter: email, LinkedIn URL, job title, company website. Each one is something enrichment could actually go and fill.
- `missing_optional()` — gaps worth showing but which must never hold a contact back: direct phone, company description, LinkedIn activity, web research.
- `recompute_status()` — *the single place* where both statuses are decided. Previously five different places in the code each ran their own copy of the rule, which meant changing the rule meant changing it five times.
- `title_drift` — true when the web says a different job title than the file. Compared loosely, so "Director, Content" and "director content" count as the same.
- `can_research` — whether to offer the Research button at all.
- `initials` — the avatar. There are no photos in a CSV and LinkedIn images cannot be reused, so it is the person's initials.

app/importer.py (528 lines) — reading the CSV

The biggest file, and for a good reason: real files are messy. What it handles:

- **Separator sniffing.** Comma, semicolon (what Excel writes in European locales), tab, or pipe. Judged over the first 20 lines, not just the first one, so a title line at the top does not decide the whole file.
- **Encoding.** UTF-8, UTF-8 with a BOM, UTF-16 (Excel's "Unicode Text"), Latin-1 — tried in a sensible order.

- **A preamble.** If the file starts with "Exported from CRM on..." it looks up to 10 rows down for the real header row, scoring each candidate by how many known fields it can recognise in it.
- **A broken header row.** Exports sometimes arrive with a whole block of column names overwritten by one repeated label. A repeated label names none of its columns, so those positions are recovered by lining the file's *trustworthy* labels up against the known Apollo column order. If fewer than 60% agree, the repair is refused rather than guessed at.
- **Alias matching.** Roughly 35 logical fields, each with a list of names it might appear under. This is why LinkedIn Sales Navigator, HubSpot, Salesforce and a plain `Name,Email` file all import.
- **Positional reading.** Values are paired with column names by *position*, so a duplicated column name can never silently swallow another column's data.
- **De-duplication.** A row carrying an Apollo Contact Id you already hold updates that contact instead of creating a second copy — and a blank cell in the fresh export does not wipe a value you already have, because a blank is not a statement that the value is gone.

`app/research.py` (443 lines) — finding things on the web

Everything that goes out to the internet, through one search API (Firecrawl). Three jobs:

1. `research_person()` — searches two ways ("Name" + company, and "Name" + job title), removes duplicates, pushes data-broker sites (ZoomInfo, RocketReach and friends) to the back because they only repeat what the CSV already says, and — crucially — throws away anything it cannot prove is about this person.
2. `find_linkedin_activity()` — searches for `site:linkedin.com/posts "Name"`. A LinkedIn post URL contains the author's profile handle, so that handle is compared *exactly* against the handle from the CSV. Only the contact's own posts survive.
3. `research_company()` — one search for what the company does, preferring a result on the company's own domain.

It also handles API failures carefully. A rejected key or an empty credit balance raises a special error that stops the whole run, because otherwise a broken setup would silently look like "25 contacts with nothing found" — which reads as thin data rather than a broken configuration. Rate limiting waits and retries (5s, 15s, 30s, 60s), honouring the server's own `Retry-After` header when it gives one.

`app/interests.py` (141 lines) — the interest chips

Collects the evidence gathered for a person, sends it to a language model, and gets back interest chips and a focus line. Two rules it enforces *in code*, not just in the prompt:

- **Every chip must cite its evidence.** A chip that comes back without an evidence field is dropped before it is saved. So no chip on the page is an unsourced guess.
- **Thin evidence gets no answer.** Fewer than two pieces of evidence and it does not even call the model. An honest empty panel beats four plausible chips invented from a job title.

Only *confirmed* evidence is used. A web finding that could not be tied to the person is displayed but never fed to the model — because a chip derived from a stranger's article would be sourced, clickable and about somebody else, which is the worst kind of error: one that looks correct.

It is written against the standard OpenAI chat format, so it runs on Groq today and a self-hosted Ollama tomorrow by changing one setting.

app/pipeline.py (167 lines) — one definition of "research"

Runs all the research steps for one contact, in order: company → web → LinkedIn → interests. It exists so that the command-line tool and the Research button do *exactly* the same thing. Two copies of this logic would drift apart, and the one used less often would be the one that rotted.

It also runs research in a background thread. A research run takes tens of seconds — longer when the API rate-limits — so holding the browser's request open would look like a hung page. Instead the contact is parked at "Researching...", the page returns immediately, and you refresh to see the result. If anything goes wrong the contact is marked "Research failed" rather than being left stuck.

app/resolve.py (312 lines) — finding a missing email or LinkedIn

This module fills in a missing email address or LinkedIn URL from the web. Its guiding rule: **it never guesses**. In particular it never builds an address from a pattern like `first.last@company.com` — that produces a plausible string, not a fact, and a plausible string in an email field is worse than an empty one because nothing downstream can tell the two apart.

What "found" means here, exactly:

- **A LinkedIn URL** is accepted when the profile handle carries the person's name *and* the indexed page text carries their employer. Two independent facts.
- **An email** is accepted when it was found published on a page that also names the person, its domain is the employer's, and its local part carries their name. Three facts, none invented. Shared mailboxes (info@, sales@, hr@ and about 25 others) are rejected, and so are free mail hosts like Gmail, because an address there cannot be tied to an employer.
- **If two different candidates pass, nothing is accepted.** Two answers is not a verified answer.

What it does *not* claim is that the mailbox exists. Proving that needs a separate verification service, which is why what it writes goes into the email status field as "found on the web" rather than borrowing Apollo's word "valid".

Note on wiring: `resolve.py` is complete and self-contained but is not currently called from the pipeline or from any page — it is available to be wired to an "enrich this contact" action.

app/db.py (79 lines) — the database connection

Opens the SQLite file at `data/peopleintel.db`, hands out database sessions, and creates the tables on startup. It also carries a small hand-written migration list: SQLAlchemy's `create_all()` creates missing *tables* but never missing *columns*, so a database made by an earlier version would break every query. The list of added columns is applied on startup, and it is deliberately kept short and repeatable rather than pulling in a full migration tool.

app/env.py (68 lines) — reading the .env file

Reads `KEY=value` lines from a `.env` file so you do not have to re-export your API keys into every new terminal — which on Windows is otherwise the whole battle. A real environment variable always wins over the file, so a value set by your host (Railway, Render, Fly) is never overwritten by a stale local file. It tolerates the two things people actually paste: a leading `export` copied from a shell snippet, and quotes copied from a dashboard. It is deliberately not a dependency — `python-dotenv` does far more than this app needs.

run_pipeline.py (170 lines) — the command-line tool

Runs research over many contacts at once, printing progress as it goes.

```
python run_pipeline.py          # everyone who still needs it
python run_pipeline.py --limit 5 # just the first five
python run_pipeline.py --slug anwar-chaudhry
python run_pipeline.py --skip-interests # research only, no model calls
python run_pipeline.py --skip-linkedin # skip activity-link searching
python run_pipeline.py --force       # re-research contacts that already have results
```

seed_research.py (309 lines) — the sample data

Loads the researched sample used in the screenshots, so you can see a populated dashboard without spending any API credits.

templates/ (12 files) — the HTML

`base.html` is the shell every page sits inside: top bar, sidebar, content area. `_partials.html` (561 lines — the largest template) holds the reusable pieces: the stat tiles, the people table, the person header, the interests panel, the LinkedIn panel, the web-findings panel, the company card and the status badges. The other ten files are one page each.

static/app.css and input.css

The styling is Tailwind, but **pre-built**. The finished `static/app.css` is committed to the project, so running the app needs no build step and no Node.js. Only if you want to change the design do you need to rebuild it — and Tailwind ships a standalone binary that needs no Node either.

7. The database explained

One SQLite file, `data/peopleintel.db`, with five tables.

The rule that shapes the whole schema: every fact the app went and fetched itself stores *where it came from* and *when*. Facts that came from your uploaded CSV store nothing extra — the file is the source.

Company

From the CSV: name, website, LinkedIn URL, industry, employee count, address, city, state, country, phone, technologies, keywords, annual revenue, total funding, latest funding, SIC and NAICS codes, Apollo account id.

Researched (and therefore with provenance): `description`, `description_source_url`, `description_fetched_at`.

Person

Identity from the CSV	first name, last name, title, seniority, department, sub-department, email, email status, LinkedIn / Twitter / Facebook URLs, direct phone, corporate phone, city, state, country, Apollo contact id
Found by the app	<code>email_source</code> , <code>email_source_url</code> , <code>email_found_at</code> , <code>linkedin_source</code> , <code>linkedin_found_at</code> — and <code>identity_note</code> , which records <i>why</i> a missing field is still missing, so the page can explain rather than show a blank
Statuses	<code>enrichment_status</code> , <code>research_status</code> , <code>status_note</code> , plus a legacy <code>status</code> field kept in step with the first one so older queries do not read a frozen value
Stale-title tracking	<code>title_observed</code> , <code>title_observed_source</code> , <code>title_observed_at</code> — the title seen on the web, kept alongside the one from the file
Derived	<code>focus_line</code> , <code>focus_generated_at</code> , <code>focus_model</code> — the "Current Focus" sentence and which model wrote it
Other	<code>slug</code> (the URL-safe name used in <code>/person/<slug></code>), <code>imported_at</code>

LinkedInActivity

One row per post. Type (post, repost, comment or tagged), URL, text, date, display rank 1–5. Crucially it also stores `added_by` — whether a human pasted it in or a search found it — plus, for searched rows, the exact query used and the fetch time, and whether authorship was confirmed.

A repost counts as activity in its own right: it is something the person chose to put in front of their network. But the words are someone else's, so reposts only ever fill slots that the person's own posts left empty.

WebFinding

One row per web result kept: title, URL, snippet, kind (article, interview, event, news or profile), rank, the query that surfaced it, when it was fetched, and `corroboration` — the plain-English reason this result was accepted as being about this person, which becomes the tooltip on screen.

Interest

The chips. Label, the evidence that produced it, whether it came from LinkedIn, the web or both, rank, and when it was generated.

8. The trust rules — the heart of the project

This section explains the decisions that make the app's output worth reading. All of them trade *quantity* for *certainty*.

The core problem: namesakes

Searching for "Laura Macleod" Nelson returned her real LinkedIn profile — and a phone listing in Maine, a freelance creative strategist, a wedding-group singer, and a 1994 academic paper about performance appraisals. **Four different people.** A name in a search result cannot be told apart from a namesake's name.

Rule 1: a web result is kept only if one of three things is true

1. The page is on the **employer's own domain** (or a subdomain of it).
2. It is the contact's **own LinkedIn profile**, matched on the handle from the CSV.
3. The text carries their **name, their job title and their employer, all together**.

Signal 3 exists because the first two alone were too narrow. "Nora Mawla. Content Marketing Manager, Corndel." is unmistakably her, and was being thrown away for the sole reason that it was not hosted on corndel.com. Three separate facts from the CSV turning up together is not a coincidence a namesake produces. What is deliberately *not* a signal: a name on its own, or a name plus an employer — "Nelson" once matched a post written by a Sean Nelson.

Anything clearing none of the three is **not stored at all**. An earlier version kept them behind an "identity unconfirmed" badge, on the theory that a labelled lead beats an empty panel. It does not: the label just hands the job of re-checking every row back to the reader, and a stranger's phone listing sitting on someone's profile is wrong whatever badge it is wearing.

The measured cost: of 85 findings re-scored under this rule, 18 were rescued by signal 3 and 36 were dropped — some of them genuinely relevant, like a conference-talk listing that simply never repeats the speaker's title and employer in its snippet. Precision was chosen over coverage on purpose. 17 of 25 contacts still have at least one finding.

Rule 2: LinkedIn posts must be proven to be theirs

A LinkedIn post URL looks like this:

```
/posts/<author-handle>_<words-from-the-post>-activity-<id>
```

Only the first part is the author. The rest is generated from the post's own text and will happily contain other people's names. So the author part — and only the author part — is compared, **exactly**, against the handle from the CSV.

Exactly, because looser tests fail convincingly:

- Matching anywhere in the URL credited a hospital's post *about* a Professor Anwar Chaudhry to our Anwar Chaudhry — his name is in the post's own text.
- The short handle `anwar-chaudhry` is itself contained inside `dr-mumtaz-anwar-chaudhry-98231b11`.
- Matching on name words credited five different Laura MacLeods' posts to ours.

A middle version showed name-matched rows behind an "unverified" badge so the panels would not be empty. It was tried and reverted: of 49 rows collected across 15 contacts, **7** were actually the right person. One

panel showed four different Christopher Carrolls; another showed a recruiter called Daryl Daley, matched because "daryl" and "speed" happen to appear in order in the phrase "Great insight, Daryl! speed with clarity".

Rule 3: no chip without evidence

Interest chips come from a language model, which means they could easily be invented. Three guards:

- Fewer than two pieces of evidence and the model is never called.
- A chip returned without an evidence field is discarded before saving.
- Unconfirmed findings are excluded from the evidence entirely — not merely ranked lower.

Rule 4: nothing is padded

Panels hold up to five items. If only one is real, one is shown. Empty panels say "Not found" or "Nothing indexed" and stay empty.

The honest consequence: coverage is uneven, and that is the data, not a bug. A senior, conference-speaking contact returns four or five solid findings. A mid-level Content Manager often returns only their own LinkedIn profile. Of the 25 sample contacts, 13 are Manager-level, so expect thin panels for most of them.

Provenance on screen

Every fact the app fetched carries a clickable source and a date, shown like this:

✓ Verified — nelson.com, fetched 24 Aug 2026

Hovering an interest chip shows which post or article produced it. Facts that came from your CSV carry no source tag — the file is the source.

9. Two statuses, not one

This is one of the more interesting design fixes in the project, and worth understanding because it changes what the badges mean.

The bug

Originally there was one status field. "Needs Enrichment" fired when research had not run, or when a company lacked a description — so it flagged 16 of 25 contacts and told you nothing about any of them. It effectively meant "something, somewhere, is missing", which is true of nearly every contact.

The fix: two independent questions

Field	The question it answers
<code>enrichment_status</code>	Do we know who this person is? Decided purely from identity fields: email, LinkedIn URL, job title, and the company's website. Values: Complete, or Needs Enrichment.
<code>research_status</code>	Has anyone looked them up? Not researched → Researching... → Researched, or Research failed.

Because they are independent, both combinations are now expressible and both are real states: *Needs Enrichment* + *Researched*, and *Complete* + *Not researched*. A failed search no longer overwrites the verdict on whether we know who the person is.

What deliberately does not block a contact

Gap	Why it is optional
Direct phone number	The Apollo export has one for nobody, so counting it would hold every single contact at "Needs Enrichment" forever.
Company description	That is a fact about the employer, and one that research fills in — not something missing from this person's record.
Web research / LinkedIn activity	Their absence means nobody has looked yet. That is research status, which is tracked separately.

Job title *is* treated as critical, because it is one of the three pillars the tool is built on and it comes from the file — so a blank one is a genuine hole. If you disagree, removing it from `missing_critical()` is a one-line change and nothing else has to move.

10. The technology used

Part	Choice	Why
Web framework	FastAPI	Fast, modern, minimal Python web framework.
Templates	Jinja2, server-rendered	Pages arrive as finished HTML. No client-side rendering.
Styling	Tailwind, pre-built	The finished CSS is committed, so no build step and no Node.js is needed to run the app.
Interactivity	Plain HTML forms	No JavaScript framework at all.
Database	SQLite via SQLAlchemy	A single file. No database server to install.
Web research	Firecrawl search API	Called over plain HTTP with <code>httpx</code> — no vendor SDK.
Language model	Any OpenAI-compatible endpoint	Groq by default; a self-hosted Ollama works by changing one setting.

The complete dependency list is six packages: `fastapi`, `uvicorn`, `jinja2`, `python-multipart`, `sqlalchemy`, `httpx`.

One deployment caveat. SQLite is fine on a host with a real disk — Railway, Render, Fly. It is *not* fine on a serverless platform like Vercel, where the filesystem is wiped between requests and your writes vanish. If you deploy serverless, point `app/db.py` at a Postgres URL (Neon and Supabase both have free tiers). Nothing else changes.

11. How to run it

First time

```
pip install -r requirements.txt

python -c "from app.db import init_db; init_db()"
uvicorn app.main:app --reload --port 8000
```

Then open **`http://127.0.0.1:8000`**, upload a CSV, and your contacts appear.

To load the sample data (no API credits spent)

```
python -c "
from app.db import init_db, SessionLocal
from app.importer import import_csv
init_db(); db = SessionLocal()
print(import_csv(db, open('data/apollo_contacts.csv', 'rb').read()))"
python seed_research.py
```

To run real research

```
python run_pipeline.py --limit 5      # try a few first
python run_pipeline.py                # everything outstanding
```

Or click the **Research** button on any contact who has not been researched yet, and refresh the page after a moment.

12. Settings and API keys

Copy `.env.example` to `.env` and fill in your keys. The server reads this file at startup — it does not watch it, so restart after editing.

Setting	What it is for
<code>FIRECRAWL_API_KEY</code>	Web research: company descriptions, the web findings panel, LinkedIn activity links. Without it, <code>run_pipeline.py</code> exits immediately. From firecrawl.dev .
<code>LLM_BASE_URL</code>	The language-model endpoint. Defaults to Groq. Point it at <code>http://localhost:11434/v1</code> for a local Ollama.
<code>LLM_MODEL</code>	Which model to use for interest detection.
<code>LLM_API_KEY</code>	The model key. Groq's free tier is enough. Ollama needs no real key.

The Settings page inside the app tells you whether each key was found and exactly which file path it looked at — useful when a key "should" be set but is not being picked up.

13. What is in the database right now

A snapshot of the current `data/peopleintel.db`:

Item	Count
Contacts (people)	25
Companies	21
Companies with a researched description	18 of 21
Web findings stored	56
LinkedIn activities stored	11
Interest chips	27
Enrichment status	25 Complete, 0 Needs Enrichment
Research status	19 Researched, 6 Not researched

There are also four backup copies of the database in `data/`, each named after the change it was taken before — before de-duplication, before the findings purge, before the general purge, and before the status split.

14. Known limits and what is not built

Deliberately not built

Feature	Why not
Email generation	Out of scope on purpose. The database has room for it.
Direct phone numbers	The Apollo export has none for any of the 25 contacts — only company switchboards. Direct lines need a paid enrichment service, so the field is shown as an optional gap and never holds anyone back.
Person photos	Not in the CSV, and LinkedIn images cannot be hotlinked or reused. Avatars are initials. Company logos come from favicons.
Login / user accounts	The header shows a fixed user. Real accounts are not implemented.
Scheduled refresh	Research is fetched once. Nothing re-runs on a schedule, so a "2 days ago" activity will read "9 days ago" a week later, and findings drift out of date.

Honest limits of what is built

- **Coverage is uneven.** Most mid-level contacts get thin panels, because most people's posts are simply not in a search index under a name a stranger can match. This is accepted, not hidden.
- **Only indexed text, not full posts.** The LinkedIn panel shows the search engine's snippet. Where no snippet exists, the row is kept as a bare link and labelled as one — and because interest chips are derived from *text*, a link-only row gives the model nothing. That is why pasting the text in by hand is still worth doing.
- **No LinkedIn URL in the CSV means no LinkedIn activity**, by design — there is no handle to check authorship against.
- **An email found by `resolve.py` is not a verified mailbox** — only an address published as theirs.
- **`resolve.py` is not yet wired into the UI** or the pipeline.

Everything above is a stated trade-off, not an accident. Each one is documented in the code with the reasoning and, in most cases, the numbers that led to the decision. If you want to trade the other way — more coverage, less certainty — the README names the exact function to loosen in each case.

15. Glossary

Term	Meaning in this project
Contact / Person	One row from your CSV — one human being.
Slug	The URL-safe version of someone's name, e.g. <code>laura-macleod</code> , used in the address <code>/person/laura-macleod</code> .
Enrichment	Filling in missing identity information about a person.
Research	Going out to the web to find what has been published about a person and their company.
Finding	One web result kept for a person.
Activity	One LinkedIn post, repost, comment or tagged mention.
Interest chip	A short 1–3 word label describing what someone currently cares about, backed by evidence.
Corroborated	The app can prove this result is about <i>this</i> person, and can say in words why.
Provenance	The record of where a fact came from and when it was fetched.
Title drift	The job title on the web disagrees with the one in your file.
Designation	Job title. One of the three pillars.
Firecrawl	The web-search API this app uses for all internet lookups.

16. Quick file reference

File	Lines	In one sentence
<code>app/main.py</code>	360	Every web page and form action.
<code>app/models.py</code>	441	The five tables and the rules attached to them.
<code>app/importer.py</code>	528	Reading messy CSV files without losing data.
<code>app/research.py</code>	443	All web lookups, and proving a result belongs to a person.
<code>app/resolve.py</code>	312	Finding a missing email or LinkedIn URL, without guessing.
<code>app/pipeline.py</code>	167	One shared definition of "research this contact".
<code>app/interests.py</code>	141	The interest chips and the focus line.
<code>app/db.py</code>	79	Database connection, table creation, small migrations.
<code>app/env.py</code>	68	Reading API keys from a <code>.env</code> file.
<code>run_pipeline.py</code>	170	Command-line research over many contacts.
<code>seed_research.py</code>	309	Loads the sample researched data.
<code>templates/_partials.html</code>	561	Every reusable panel, tile, table and badge.
<code>templates/base.html</code>	80	The page shell: top bar, sidebar, content area.
<code>templates/*.html</code>	~590	Ten pages: home, upload, people, person, companies, company, enrichment, research, reports, settings.
<code>static/app.css</code>	—	Pre-built Tailwind stylesheet, committed so no build step is needed.
<code>requirements.txt</code>	6	The entire dependency list.
<code>.env.example</code>	—	Template for your API keys.